AI Mock Interviewer — Project Spec

Oct 6, 2026 · @Hibah

Overview

A voice-based mock interviewer for any job: the user uploads a résumé and job description, picks a length (10, 30, 45 or 60 minutes, or custom), talks to an AI interviewer, and gets a score plus concrete advice on what to say instead.

- **Who it's for:** anyone job-hunting, in any field — software, sales, marketing, media, business, healthcare, retail, and more.
- **What makes it different:** role-specific interview formats (including role-plays), time-budgeted sessions, rewritten "stronger answers" in the user's own voice, and a story bank that builds up across sessions.
- **Cost rule:** everything runs on free tiers or open-source software. The only paid item is the developer's Claude Pro subscription. Any feature that would cost money is left out or given a free fallback.
- **Framing:** scores are practice feedback, not a hiring assessment or certification.

Free tech stack

Every layer has a free option; the right column lists what to avoid because it would start costing money.

Layer	Free choice	Avoid (costs money)
Frontend	React + Vite + Tailwind, LiveKit React components	—
In-browser tools	Monaco (code editor), Excalidraw (whiteboard), MediaPipe (body language), all running in the user's browser	Server-side video processing
Backend API	Flask (Python)	—
Voice agent	LiveKit Agents (Python, open source)	Vapi, Retell, or other per-minute platforms

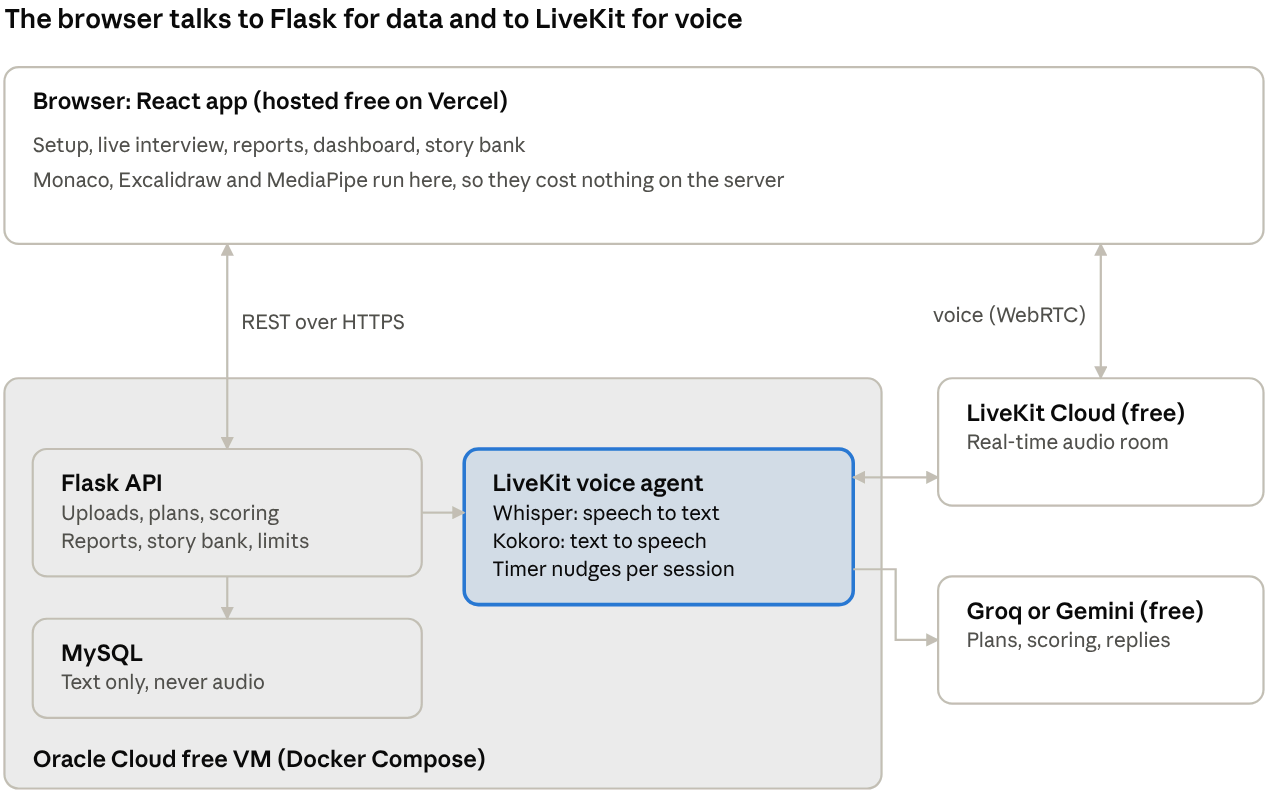
Layer	Free choice	Avoid (costs money)
Real-time audio	LiveKit Cloud free tier, or self-hosted LiveKit server	Twilio or any phone numbers
LLM	Groq or Gemini free tier, behind a swappable provider setting	Paid LLM plans
Speech-to-text	Local Whisper (faster-whisper) by default; Deepgram signup credits as a bonus	Paid speech-to-text once credits run out
Text-to-speech	Kokoro or Piper, running locally (multiple voices for panel mode)	ElevenLabs or other paid voices
Database	MySQL	Managed paid databases
PDF parsing and export	pypdf / pdfplumber to read; WeasyPrint or browser print to export	—
Hosting	Oracle Cloud Always Free virtual machine (backend, agent, database); Vercel or Netlify (frontend)	Paid hosting or anything that auto-upgrades
Domain and HTTPS	Free subdomain (DuckDNS) with Caddy for automatic HTTPS	Paid domain (optional)
Coding help	Claude Code with Claude Pro (the one paid item)	—

Free-tier notes:

- Oracle's free ARM allowance is now 2 cores and 12 GB of RAM. Set a $1 budget alert so nothing becomes billable by accident.
- Groq, Gemini and Deepgram free tiers have rate limits or finite credits, so the app enforces per-user daily limits (see Privacy, safety, and free-tier guards).

Architecture

The browser sends data requests to Flask and streams voice through LiveKit; one free Oracle VM runs everything server-side.



system architecture · 6 components, all on free tiers

How a session flows:

1. The user uploads a résumé and job description; Flask parses them and asks the LLM for a role analysis, question plan, and rubric.
2. Flask saves the plan in MySQL and creates a LiveKit room with the plan attached to the session.
3. The voice agent joins the room, loads the plan, and runs the interview: Whisper turns speech into text, the LLM writes replies, Kokoro speaks them.
4. Flask's timer sends section nudges to the agent; the coding editor and whiteboard stream their contents over LiveKit data channels.
5. When the session ends, Flask scores each answer in the background, writes recommendations, updates the story bank, and builds the report.

Core features

The core loop is setup → interview → report → retry, with progress tracked across sessions.

Setup

- Upload a résumé (PDF or text) and a job description (paste or PDF).
- Choose a duration: 10, 30, 45 or 60 minutes, or a custom slider from 5 to 60.
- Choose a mode: behavioral, technical, case, role-play, system design, coding, or a full mixed loop. Only modes that fit the job are offered.
- Choose difficulty (entry, mid, senior) and interviewer persona (friendly, neutral, tough).
- Preview the generated question plan, with minutes per section, and edit it before starting.

During the interview

- Adaptive follow-ups that probe vague answers.
- Backend timer with section nudges ("4 minutes left in this section") and a graceful wrap-up; the agent never cuts off mid-sentence.
- Bonus questions from a reserve list if the candidate finishes early; polite redirects if they ramble.
- "Repeat the question," "Can I have a hint?" (hints are noted in the score), pause and resume.
- Natural interruption handling: the agent stops talking when the user cuts in.
- Visible countdown and current section, with an option to hide it.
- Live captions and a text-only fallback mode.
- Coding round (Monaco) and whiteboard (Excalidraw) that stream to the agent, shown only for roles that need them.

Report

- Score per question against the role's rubric, plus an overall "hire / lean hire / no hire" verdict with reasoning.
- Transcript with weak moments highlighted.
- Speech analytics from transcript timestamps: filler words, pace, long pauses, time per answer versus budget.
- STAR-structure analysis for behavioral answers.
- Report builds question by question in the background, so it fills in progressively.
- PDF export and a shareable link.

Recommendations (what you could have said)

- "What you said" next to "A stronger version," rewritten in the user's voice using only facts from their résumé or answers.
- Never invents achievements: missing details become placeholders such as "[add the % increase you achieved]".
- Highlighted changes with reasons: added, cut, reframed.
- Checklist of points a strong answer would have covered, based on the rubric.
- Delivery tips (fillers, length, trailing off) and tips specific to the field.
- Top 3 takeaways at the top of the report.
- "Try again" on any question, with the before-and-after score shown.

Progress

- Dashboard of scores over time, broken down by competency.
- Weak-spot sessions generated from past reports.
- Personal question bank with spaced repetition: missed questions come back later.
- Practice streaks and goals.

Any-job support

The app supports any job by mixing a small set of reusable interview format modules, chosen per job from the job description.

Step 1: role detection. The LLM reads the job description and returns the field, seniority, key competencies, and the formats that fit.

Step 2: format modules. Each module is a reusable prompt and agent behavior:

Module	What happens	Typical fields
Behavioral (STAR)	"Tell me about a time…" with follow-ups	Every job
Résumé deep-dive	Probes specific résumé lines and gaps against the job description	Every job
Technical Q&A	Knowledge questions for the field	Software, data, finance, engineering

Module	What happens	Typical fields
Coding	Live problem in Monaco; agent discusses the code	Software, data
System design	Whiteboard in Excalidraw; agent reviews the design	Software
Case	Market sizing, launch plans, metrics problems	Business, consulting, marketing, product
Role-play	Agent becomes a character (skeptical buyer, angry customer, recruiter)	Sales, customer service, retail, hospitality
Scenario	"What would you do if…" judgment questions	Healthcare, teaching, management, trades
Portfolio walkthrough	User explains past work; agent probes decisions	Marketing, media, design

Adding a new field means choosing a new mix of modules, not writing new code.

Step 3: rubrics per role. Each session gets a rubric built from the job description's competencies (sales: rapport, objection handling, closing; nursing: patient safety, communication, judgment), plus universal criteria (structure, clarity, relevance). The rubric is shown in the report so scores feel fair.

Role-play rule: the agent clearly announces when it switches into a character and when it switches back to debrief.

Accuracy caveat: LLM scoring is weaker in specialized or licensed fields (healthcare, law, accounting). Those reports carry a short disclaimer that feedback is for practice only.

Rollout order: software first (easiest to judge quality), then sales and customer service role-plays, then all other fields.

Signature features

These six features set the project apart; the story bank is the headline.

1. Story bank

- After every session, the LLM extracts reusable stories from the user's answers ("fixed the payment bug," "calmed an angry client") with a title, a STAR summary, and the competencies each one demonstrates.
- Duplicate stories across sessions are merged, keeping the strongest version.
- A coverage view maps stories to competencies and flags gaps: "No story for conflict or failure."
- Before an interview, the app suggests which stories fit the job description.
- Users can edit, rename, or delete any story.
- Free: extraction is one LLM call per session, run in the background.

2. Post-interview debrief (no recordings, ever)

- After a real interview, the user recalls the questions and their answers from memory, by typing or by speaking.
- **Strictly no recordings:** the app never accepts audio or video uploads of a real interview. Spoken recall is transcribed live and the audio is discarded immediately; only the user's text is saved.
- The app rebuilds the interview, scores each answer, writes stronger versions, and predicts likely next-round questions.
- One-click "Practice the next round" builds a session from the debrief.
- A visible notice explains why: recording someone without consent is illegal in some places.

3. Reverse mode (you're the interviewer)

- The AI plays a candidate for the user's chosen job: strong, average, weak, or exaggerating their résumé.
- The user asks questions, then makes a hire / no-hire decision with reasons.
- The app reveals the candidate's hidden profile and scores the user's questions and judgment.
- Useful for job seekers (hearing weak answers from the outside) and new managers practicing hiring.

4. Pressure mode

- Difficulty levels that add interruptions, a skeptical interviewer, time limits ("30 seconds"), curveball questions, and mid-answer pushback ("That's not what I asked").
- Tough but always respectful: no insults or personal attacks, enforced in the system prompt.
- Tracks a composure score: how much answer quality holds up under pressure compared with the user's normal sessions.
- User can stop or lower the pressure at any time.

5. Peer practice

- Two people join one LiveKit room by invite link; one interviews, one is the candidate, then they can swap.
- The AI listens silently, scores the candidate, and coaches the interviewer on question quality.
- Both people must accept a consent screen before audio is transcribed; nothing is recorded or stored as audio.
- The person interviewing gets suggested questions from the candidate's job description.
- Cheap on free tiers: the AI only listens and summarizes, so it uses fewer LLM tokens than a normal session.

6. Job description decoder

- Reads between the lines of a job description: likely team size, pace, real priorities, red flags, and green flags.
- Example: "'Stakeholders' appears 6 times, so expect communication questions."
- Lists the questions likely to be asked and the competencies to prepare.
- Clearly labeled as interpretation, not fact.
- Works without signing in, as a shareable entry point to the app.

Extra features

Seven smaller features, all free because they run in the browser or need only a single LLM call.

Feature	What it does	Why it stays free
Body language feedback	Optional camera check of eye contact, posture, and fidgeting	MediaPipe runs in the browser; video never leaves the device
Panel interview	Two or three interviewers (hiring manager, peer, HR) with different voices	Kokoro includes multiple local voices
Résumé gap analysis	Shows which job requirements the résumé covers and which questions the gaps will trigger	One LLM call
Company research mode	User pastes the company's website link; the app reads the public About or Careers page and asks "Why us?" questions	Simple page fetch plus one LLM call
Negotiation role-play	Agent plays a recruiter making a low offer; scored on anchoring, justification, and staying calm	Same voice pipeline
Questions-for-them coach	Helps prepare smart questions to ask at the end and rates the ones asked	One LLM call
Cheat sheet and thank-you email	One-page PDF of best stories, key numbers, and weak spots; a drafted thank-you email from the session	One LLM call plus PDF export

Database schema

Thirteen MySQL tables cover every feature; flexible LLM output (plans, rubrics, analytics) is stored as JSON columns. No table stores audio or video.

Table	Key columns	Notes
users	id, email, password_hash, name, created_at	Auth via Flask + JWT

Table	Key columns	Notes
documents	id, user_id, kind (resume / job_description), raw_text, parsed_json, expires_at	Original files deleted after parsing
jd_analyses	id, document_id, field, seniority, competencies_json, formats_json, decoder_json	Shared by setup and the decoder
sessions	id, user_id, resume_id, jd_id, mode (standard / reverse / pressure / peer / debrief), duration_min, difficulty, persona, pressure_level, plan_json, rubric_json, status, started_at, ended_at	One row per interview
questions	id, session_id, section, order_no, text, budget_sec, is_bonus	The question plan
answers	id, question_id, transcript, started_ms, ended_ms, attempt_no, hints_used	attempt_no supports "Try again"
evaluations	id, answer_id, score, rubric_scores_json, missing_points_json, stronger_version, changes_json, delivery_json	Recommendations live here
reports	id, session_id, overall_score, verdict, top_takeaways_json, analytics_json, share_token	share_token for public links
stories	id, user_id, title, star_json, competencies_json, source_answer_ids_json, strength	Story bank
debriefs	id, user_id, company, role, round, recalled_qa_json, analysis_json	Text only, no recordings
peer_rooms	id, host_user_id, guest_user_id, invite_code, consent_host, consent_guest, session_id	Peer practice
question_bank	id, user_id, question_text, competency, last_score, next_review_at	Spaced repetition
usage_limits	user_id, date, minutes_used, llm_calls, sessions_count	Free-tier guard

Privacy, safety, and free-tier guards

The app stores text only, asks for consent before anyone else is heard, and caps usage so free tiers never run out.

Privacy

- No audio or video is ever stored. Speech is transcribed live and discarded.
- No uploads of real-interview recordings are accepted (debrief is recall-only).
- Camera analysis runs entirely in the browser and is off by default.
- Résumés and job descriptions auto-delete after 30 days unless the user saves them.
- "Delete my data" removes everything for that user in one click.
- Shared report links can be revoked.

Safety

- Pressure mode and role-plays stay respectful: no insults, slurs, or personal attacks.
- Stronger-answer rewrites never invent experience; placeholders mark missing facts.
- Disclaimers on scores for licensed and specialized professions.
- The JD decoder labels its output as interpretation.

Free-tier guards

- Daily caps per user, tracked in usage_limits. Suggested starting caps: 60 interview minutes, one 60-minute session, 5 decoder runs.
- Provider fallbacks: when an LLM or speech API hits its limit, switch to the next free provider or to local models.
- Defaults are 10 and 30 minutes to save quota.
- Oracle budget alert at $1; no paid add-ons enabled in any account.
- A small admin page shows daily usage against each free tier's limits.

Build phases with Claude Code

Build in ten phases, one feature per Claude Code session, committing to git after each. Each phase ends with something that works on its own.

1. **Foundation:** repo layout, `CLAUDE.md`, Flask app, MySQL schema, sign-up and login, React shell.

2. **Text-only brain:** résumé and job description upload → JD analysis → question plan with time budgets → rubric. Test it with typed answers before adding voice.
3. **Voice interview:** LiveKit Agents with local Whisper and Kokoro, the plan injected per session, captions, interruptions.
4. **Timer and sections:** backend clock, section nudges, wrap-up, bonus questions, all four durations plus custom.
5. **Report and recommendations:** scores, stronger versions, missing points, delivery analytics, top 3 takeaways, "Try again."
6. **Any-job support:** role detection, format modules, role-play, per-role rubrics. Software first, then sales and customer service.
7. **Progress and story bank:** dashboard, story extraction and coverage view, question bank with spaced repetition.
8. **Signature modes:** JD decoder, debrief, reverse mode, pressure mode, peer practice.
9. **Extras:** coding round, whiteboard, panel, body language, gap analysis, company research, negotiation, cheat sheet, thank-you email.
10. **Deploy and harden:** Oracle VM with Docker Compose and Caddy, frontend on Vercel, usage caps, evaluation harness (AI candidates across fields), privacy controls.

Working within Claude Pro limits (Sonnet only, roughly 10 to 40 Claude Code prompts every 5 hours):

- Keep `CLAUDE.md` current so each session starts with full context.
- Use plan mode for each phase, approve the plan, then implement.
- Ask for small, testable changes; review and commit before the next prompt.
- Do repetitive work (prompt tuning, copy edits) by hand to save quota for real coding.